

Αναφορά Project Φάση E2

Ομάδα 16: Σπυρίδων Σιαχάμης - 5503, Άγγελος Σαλλούφι - 5245

Μέρος A: Βάση δεδομένων

Στο πρώτο μέρος της εργασίας δημιουργήσαμε τη βάση δεδομένων του συστήματος με βάση το σχεσιακό μοντέλο που είχε σχεδιαστεί στη φάση E1. Η βάση υλοποιήθηκε στο MySQL container του template και η διαχείρισή της έγινε μέσω phpMyAdmin. Αντί να χρησιμοποιήσουμε τον αρχικό δοκιμαστικό πίνακα events που υπήρχε ήδη στο template, τον διαγράψαμε και φτιάξαμε το δικό μας σχήμα βάσης ώστε να ταιριάζει καλύτερα με το σύστημα ταξιδιωτικής ανακάλυψης που σχεδιάσαμε.

Οι βασικοί πίνακες που χρησιμοποιήθηκαν για τα κομμάτια της υλοποίησης είναι οι users, events, content και participation. Στην υλοποίηση τα trips αποθηκεύονται στον πίνακα events, επειδή κάθε εκδρομή/δραστηριότητα του συστήματος αντιμετωπίζεται ως event.

Ο πίνακας users αποθηκεύει τα βασικά στοιχεία των χρηστών του συστήματος. Περιλαμβάνει πεδία όπως user_id, username, password, first_name, last_name, email και role. Το user_id είναι το πρωτεύον κλειδί του πίνακα και χρησιμοποιείται για να συνδεθεί ο χρήστης με άλλες λειτουργίες του συστήματος. Το πεδίο role χρησιμοποιείται για να ξεχωρίζει αν ο χρήστης είναι απλός εκδρομέας ή δημιουργός εκδρομών.

Ο πίνακας events αποθηκεύει τις εκδρομές που εμφανίζονται στην πλατφόρμα. Περιλαμβάνει πεδία όπως event_id, trip_creator_id, title, event_date, event_time, type, status, max_participants, price, description, schedule, address_id, rating και preview_image. Το event_id είναι το πρωτεύον κλειδί, ενώ το trip_creator_id είναι ξένο κλειδί που δείχνει ποιος δημιουργός έχει φτιάξει την εκδρομή. Το preview_image κρατάει το path της εικόνας που εμφανίζεται στο UI, για παράδειγμα /images/knossos.png. Η βάση δεν αποθηκεύει την ίδια την εικόνα, αλλά μόνο τη διαδρομή της. Οι εικόνες βρίσκονται στον φάκελο public/images του frontend.

Ο πίνακας event_content αποθηκεύει το περιεχόμενο που συνδέεται με μια εκδρομή. Περιλαμβάνει πεδία όπως content_id, user_id, event_id, title, content_type, text_description,

approval_status, approved_by, approval_date, scheduled_time και publish_moment. Το content_id είναι το πρωτεύον κλειδί, ενώ το event_id δείχνει σε ποια εκδρομή ανήκει το περιεχόμενο. Το content_type χρησιμοποιείται για να δηλώσει αν το περιεχόμενο είναι φωτογραφία, highlight ή video link. Για τα βίντεο αποθηκεύεται σύνδεσμος και όχι αρχείο βίντεο, όπως είχε προβλεφθεί και από τη σχεδίαση.

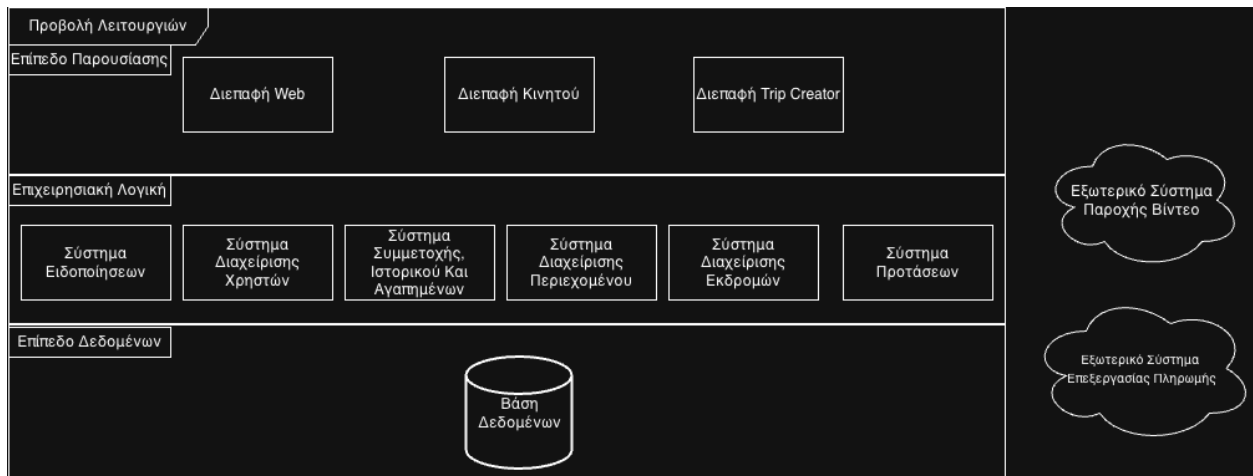
Ο πίνακας participations αποθηκεύει τις συμμετοχές των χρηστών σε εκδρομές. Περιλαμβάνει πεδία όπως participation_id, user_id, event_id, payment_price, participation_date και rating. Το participation_id είναι το πρωτεύον κλειδί, ενώ τα user_id και event_id είναι ξένα κλειδιά προς τους πίνακες users/excursionists και events. Με αυτόν τον πίνακα μπορεί να φανεί σε ποιες εκδρομές έχει δηλώσει συμμετοχή ένας χρήστης. Επίσης, από αυτές τις εγγραφές μπορεί να προκύψει και το ιστορικό εκδρομών ενός χρήστη.

Εκτός από τους παραπάνω βασικούς πίνακες, δημιουργήθηκαν και βοηθητικοί πίνακες όπως addresses, favourites, notifications, keywords, event_keywords και event_statistics. Αυτοί χρειάζονται για να καλύπτουν καλύτερα τις λειτουργίες που είχαν οριστεί στη φάση E1, όπως τοποθεσίες, αγαπημένα, ειδοποιήσεις, λέξεις-κλειδιά και στατιστικά. Για να μπορεί να δοκιμαστεί η εφαρμογή, προσθέσαμε και αρχικά δεδομένα με INSERT εντολές, όπως χρήστες, εκδρομές, εικόνες εκδρομών, συμμετοχές και ειδοποιήσεις.

Σε σχέση με τη προηγούμενη φάση η μόνη αλλαγή που κάναμε είναι να βάλουμε το event keywords και το event statistics. Τα άλλα έμειναν όπως πριν.

Μέρος Β: Μικροϋπηρεσίες

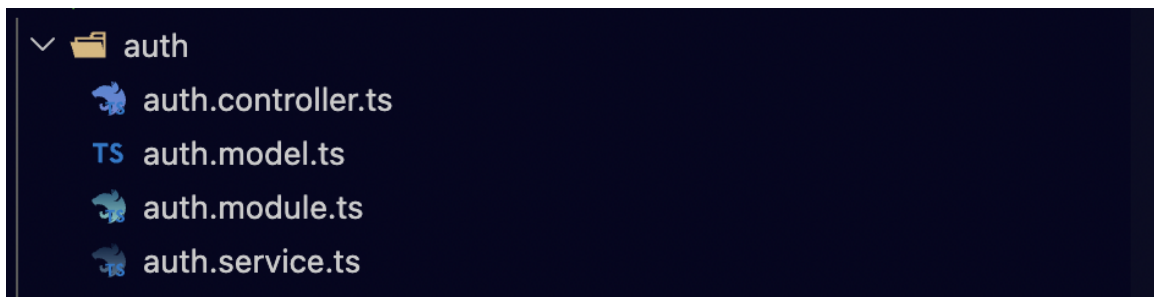
Στην φάση E2 υλοποιήσαμε τις ακόλουθες μικροϋπηρεσίες από τον αρχιτεκτονικό σχεδιασμό της φάσης E1. Το διάγραμμα της E1 παρουσιάζει τις συνολικές μικροϋπηρεσίες που σχεδιάστηκαν για το Πληροφοριακό Σύστημα στο επίπεδο Επιχειρησιακής Λογικής:



Για τις απαιτούμενες σελίδες που ζητήθηκαν στην φάση E2, υλοποιήθηκαν οι παρακάτω:

Σύστημα Διαχείρισης Χρηστών/User Management Module

Το User Management Module στο σύστημα μας αποτελεί το auth folder στο backend (backend/src/auth). Αποτελείται από τα ακόλουθα αρχεία:



Στο model περιγράφονται τα interfaces που χρησιμοποιούνται στο User Management module. Αυτά είναι:

```

1  export interface user {
2      username: string;
3      password: string;
4  }
5
6  export interface AuthUser {
7      user_id: number;
8      username: string;
9      role: string;
10 }

```

Στον auth.controller, υπάρχουν τα ακόλουθα API endpoints:

`@Post('login')`

Αποτελεί ένα HTTP POST request που αναλαμβάνει το την διαδικασία ταυτοποίησης του χρήστη, όταν αυτός συνδεθεί στο σύστημα. Όταν αυτό το endpoint τρέξει, χρησιμοποιεί το `authenticateUser` από το `auth.service` και επιστρέφει ένα μήνυμα επιτυχίας σύνδεσης και δημιουργεί το κατάλληλο session για τον χρήστη μαζί με το όνομα του και το είδος χρήστη που είναι.

`@GET('session')`

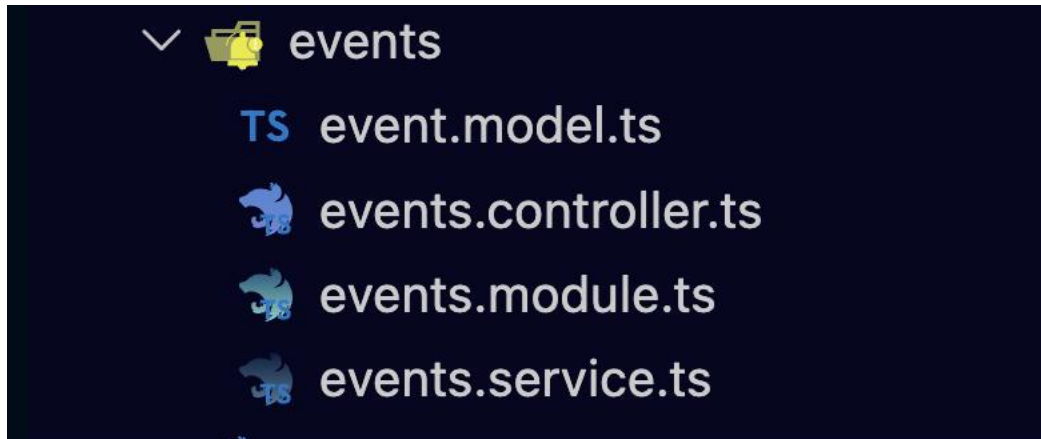
Αποτελεί ένα HTTP GET request το οποίο αναλαμβάνει τον έλεγχο της κατάστασης σύνδεσης του χρήστη (session validation). Κατά την κλήση του, το endpoint ελέγχει αν υπάρχει ενεργό session και αν αυτό περιέχει έγκυρα στοιχεία χρήστη. Σε περίπτωση που ο χρήστης δεν είναι συνδεδεμένος, επιστρέφει ένα `UnauthorizedException`. Αν η επαλήθευση είναι επιτυχής, επιστρέφει ένα αντικείμενο που επιβεβαιώνει τη σύνδεση, μαζί με το όνομα και τον ρόλο του χρήστη, επιτρέποντας στο UI να διαχειριστεί τα δικαιώματα πρόσβασης της σελίδας.

`@Post('logout')`

Αποτελεί ένα HTTP POST request το οποίο αναλαμβάνει τον τερματισμό της σύνδεσης του χρήστη. Κατά την κλήση του, το endpoint ελέγχει αν υπάρχει ενεργό session και αν αυτό περιέχει έγκυρα στοιχεία χρήστη. Σε περίπτωση που ο χρήστης δεν είναι συνδεδεμένος, επιστρέφει ένα `UnauthorizedException`. Αν η επαλήθευση είναι επιτυχής, καλεί το `logoutUser(session)` από το `auth.service` που καταστρέφει το session του χρήστη και επιστρέφει ένα μήνυμα επιτυχίας με `success: true` στο frontend.

Σύστημα Διαχείρισης Εκδρομών/Event Management Module

Το Event Management Module στο σύστημα μας αποτελεί το events folder στο backend (backend/src/events). Είναι υπεύθυνο για την επιχειρησιακή λογική και τη διαχείριση των εκδρομών. Αποτελείται από τα ακόλουθα αρχεία:



Στο model περιγράφεται το interface του event. Χρησιμοποιείται για εξυπηρέτησι τις πληροφορίες που παρέχει ο trip creator απο το nextjs frontend (add-event page) και για να εισαχθούν οι πληροφορίες για τα SQL queries:

```
1  export interface Event {
2      event_id?: number;
3      trip_creator_id: number;
4      title: string;
5      event_date: string;
6      event_time: string;
7      type: string;
8      status: 'current' | 'upcoming' | 'completed';
9      max_participants: number;
10     price: number;
11     description?: string;
12     schedule?: string;
13     address_id?: number;
14     rating?: number;
15     preview_image?: string;
16 }
```

Στο events.controller, υπάρχουν τα ακόλουθα API endpoints:

@Get()

Αποτελεί ένα HTTP GET request το οποίο αναλαμβάνει την άντληση και προβολή όλων των διαθέσιμων εκδρομών στην πλατφόρμα. Όταν αυτό το endpoint καλείται, χρησιμοποιεί τη μέθοδο `findAll` από το `events.service` η οποία επιστρέφει ολόκληρη τη λίστα με τις εκδρομές στο frontend, καλύπτοντας τις ανάγκες της σελίδας προβολής διαθέσιμων εκδρομών.

@Get('/:id')

Αποτελεί ένα HTTP GET request το οποίο αναλαμβάνει την άντληση των στοιχείων μιας συγκεκριμένης εκδρομής με βάση το μοναδικό της αναγνωριστικό (`id`) που περνάει ως παράμετρος. Όταν αυτό το endpoint τρέξει, μετατρέπει το `id` σε ακέραιο και χρησιμοποιεί τη μέθοδο `getById` από το `events.service`. Στη συνέχεια, επιστρέφει τα αναλυτικά στοιχεία της επιλεγμένης εκδρομής στο frontend, καλύπτοντας τις ανάγκες της σελίδας προβολής επιλεγμένης εκδρομής.

@Post('/add')

Αποτελεί ένα HTTP POST request το οποίο πραγματοποιεί τη δημιουργία και καταχώρηση μιας νέας εκδρομής στο σύστημα από τους Trip Creators. Κατά την κλήση του, το endpoint ελέγχει αρχικά αν υπάρχει ενεργό session και αν ο χρήστης είναι συνδεδεμένος σε αντίθετη περίπτωση, επιστρέφει ένα `UnauthorizedException`. Επιπλέον, χρησιμοποιεί τον `FileInterceptor` για να δεχτεί την εικόνα της εκδρομής (`imageFile`), περνώντας την απο ελέγχους για το μέγεθος και τον τύπο του αρχείου χρησιμοποιώντας `regex`. Τέλος, αντλεί το `user_id` του δημιουργού από το session και καλεί τη μέθοδο `create` του `events.service` για να αποθηκευτεί η εκδρομή μαζί με το αρχείο της στη βάση δεδομένων.

Μέρος Γ: Διεπαφή χρήστη (UI)

Γ1. Σχεδίαση υψηλής πιστότητας (high-fidelity)

Σε αυτή την φάση, καθώς μας ζητήθηκαν μακέτες υψηλής πιστότητας, κάτι που δεν είχαμε στην φάση E1, καταλήξαμε στον σχεδιασμό νέων, καλύτερων μακετών σε σχέση με τις προηγούμενες. Αυτή η νέα έκδοση είναι πιο όμορφη και ταιριάζει καλύτερα στο πώς θέλαμε να παρουσιάσουμε το σύστημα αυτό. Παρακάτω παρουσιάζονται οι νέες μακέτες:

Σελίδα Σύνδεσης χρηστή στο σύστημα (login-page):



Travel
Heraklion

Welcome To Heraklion
Please Sign in to your account

Username

Password

Show Password

Login

Don't have an account? [Sign up here](#)

Η μακέτα αυτή δημιουργήθηκε με βάση το HTML5 layout, χρησιμοποιώντας το tag aside για τη φόρμα σύνδεσης (login form) στα αριστερά. Επιλέξαμε μια χαρακτηριστική παραλία του νομού Ηρακλείου ως το κύριο φόντο της σελίδας αυτής, καθώς για σχεδιαστικούς λόγους επιλέχθηκε ο νομός Ηρακλείου ως ο βασικός νομός υλοποίησης και παρουσίασης της πλατφόρμας, υιοθετώντας έτσι ένα αντίστοιχο τοπικό θέμα.

Σελίδα Προβολής διαθέσιμων εκδρομών (events-page):

Travel
Heraklion

[Home](#)[Events](#)[Favourites](#)[History](#)[Notifications](#)[Add Event](#)[Profile](#)[Settings](#)



Browse Available Events

Recommended for you



Heraklion, Knossos Tour

Price: 135€



Guided Day Tour in Heraklion

Price: 25€



Knossos Palace and Historical Heraklion city tour, with old market

Price: 50€



Greek Food Walking Tour in Heraklion

Price: 10€

Today's top pick

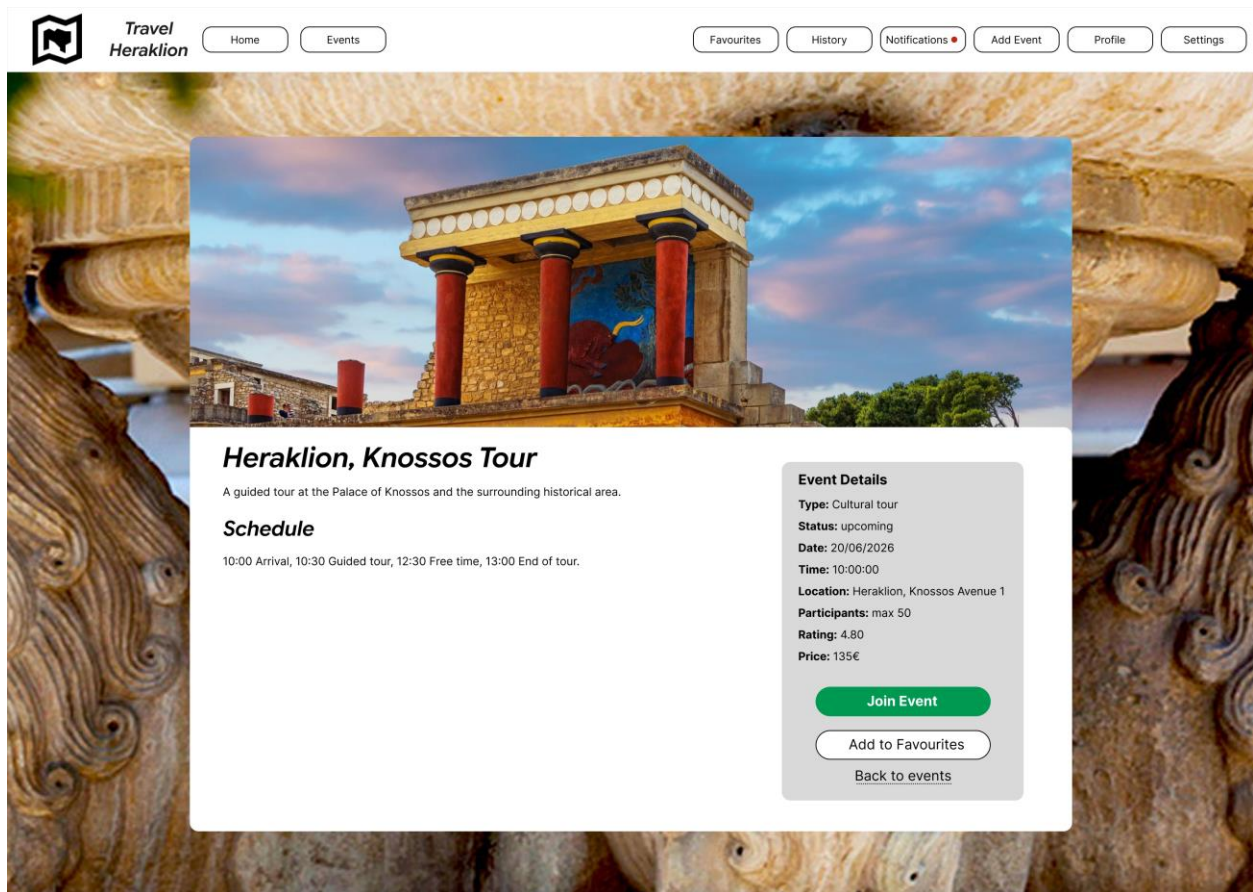


Greek Food Walking Tour in Heraklion

A short walking food tour through Heraklion, including local products and traditional tastes.

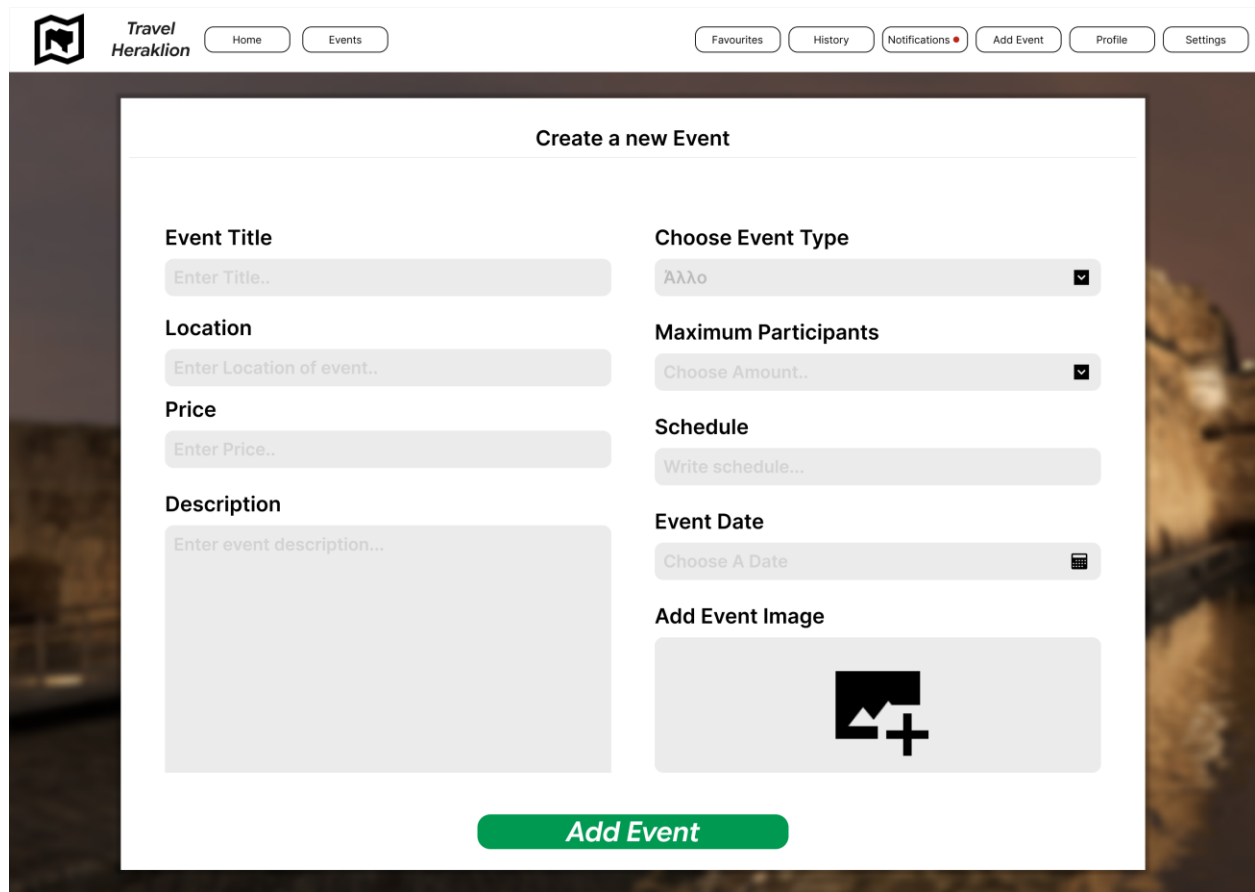
Στα υπόλοιπα δεν ακολουθήσαμε το HTML5 layout ακριβώς. Στο events page βάλαμε Σαν background τα λιοντάρια απο το κέντρο του Ηρακλείου, μιας και η εφαρμογή βασίζεται στο Ηράκλειο. Απο κάτω έχει ένα carousel με τα recommended for you για τον νομό του Ηρακλείου. Κάτω κάτω βρίσκεται το Today's top pick το οποίο προτείνει μια απο τις παραπάνω επιλογές με εκτενείς περιγραφή για πουλήσει την εκδρομή στον χρήστη.

Σελίδα Προβολής επιλεγμένης εκδρομής (view_event-page):



Σε αυτή τη σελίδα έχουμε σαν background πάλι τα λιοντάρια (μιας και είναι το expand απο τις εκδρομές του events page) Έχει ένα κουτί στο κέντρο όπου πάνω βρίσκεται το image του excursion, κάτω αριστερά έχει το όνομα, τη περιγραφή και το πρόγραμμα του. Στην κάτω δεξιά μεριά του κουτιού βρίσκεται ένα box με τα details του της εκδρομής (Type, Status, Date, Time, Location, Participants, Rating, Price) και 3 κουμπιά τα join event, add to favourites και back to events.

Σελίδα Προσθήκης εκδρομής (addevent-page):



Travel Heraklion Home Events Favourites History Notifications Add Event Profile Settings

Create a new Event

Event Title Enter Title..	Choose Event Type Άλλο ☒
Location Enter Location of event..	Maximum Participants Choose Amount.. ☒
Price Enter Price..	Schedule Write schedule...
Description Enter event description...	Event Date Choose A Date
	Add Event Image 

Add Event

Στην σελίδα αυτή, ακολουθήσαμε ένα απλό Grid layout 1x2, σε κάθε grid item υπάρχει flow auto / flex με column direction για να στοιχηθούν όσο πιο ομοιόμορφα γίνονται τα input fields. Χρησιμοποιήσαμε ένα εικονίδιο εικόνας για να παρουσιάσουμε στον χρήστη με οπτικό τρόπο τον σκοπό του field αυτού. Ως φόντο της σελίδας χρησιμοποιήθηκε ένα 3D render νυχτερινής λήψης του Ενετικό Φρούριο Ηρακλείου, διατηρώντας τη σχεδιαστική συνέπεια με το τοπικό θέμα της πλατφόρμας.

Γ2. Κατασκευή σελίδων και επικοινωνία με Μικροϋπηρεσίες

Στο frontend της εφαρμογής επιλέχθηκε μια υβριδική αρχιτεκτονική (Hybrid Architecture), συνδυάζοντας το Server-Side Rendering (SSR) και το Client-Side Rendering (CSR) ανάλογα με τις ανάγκες της κάθε σελίδας. Για τη διαδικασία της ταυτοποίησης (Authentication), χρησιμοποιήθηκε επιπλέον ο μηχανισμός Sessions της Express στην πλευρά του backend.

Σελίδα Σύνδεσης χρήστη στο σύστημα (login-page)

Η σελίδα αυτή αποτελεί το Home Page της πλατφόρμας και βρίσκεται στο αρχείο `pages/index.tsx`. Χρησιμοποιεί το Client-Side Rendering, καθώς δεν απαιτείται να φορτώσει δυναμικά δεδομένα κατά την αρχική της εκκίνηση. Το κουμπί Sign up here είναι εμφανισιακό, καθώς δεν χρειάστηκε η δημιουργία σελίδας εγγραφής (sign-up page) σε αυτή τη φάση.

Το NavBar είναι ένα reusable React component που βρίσκεται στο `components/NavBar.tsx`. Η εμφάνισή του αλλάζει με βάση το είδος του χρήστη που συνδέεται στο σύστημα. Αυτό το καταφέρνουμε με ένα global AuthContext που μεταφέρει το state του στα children της σελίδας. Είναι nested στο `_app.tsx` για να λειτουργεί ως το global layout που χρησιμοποιούν όλες οι σελίδες του συστήματος. Με αυτόν τον τρόπο, εξασφαλίζουμε έναν κοινό τρόπο διαχείρισης ώστε να έχουμε το ίδιο logged-in state σε κάθε σελίδα και να αλλάζει αυτόματα το NavBar σε όλο το σύστημα, ακόμα και αν αποφασίσουμε να δημιουργήσουμε μια ολοκαίνουργια σελίδα.

Η σελίδα σύνδεσης επικοινωνεί με το backend μέσω της συνάρτησης `submitHandler`, η οποία καλεί την `authenticate`. Η `authenticate` κάνει fetch στο `localhost:5000`, όπου βρίσκεται το backend container μας. Χρησιμοποιεί τη μέθοδο POST με την ιδιότητα `credentials: "include"`, ώστε να επιτρέπει στον browser να αποθηκεύει το session cookie που εκδίδει το backend και να το υποβάλλει αυτόματα σε κάθε επόμενο request. Στο σώμα (body) του fetch request περιλαμβάνονται οι πληροφορίες που εισήγαγε ο χρήστης, οι οποίες μετατρέπονται σε FormData και γίνονται type cast σε user type. Στην πλευρά του User Authentication Module, το backend επιβεβαιώνει τα στοιχεία του χρήστη, δημιουργεί ένα νέο Session και αποθηκεύει τον ρόλο του, το ID του και θέτει το `isLoggedIn` flag σε true. Όταν επιστρέφεται το μήνυμα επιτυχίας στο frontend, το AuthContext αλλάζει την κατάσταση του NavBar ώστε να ταιριάζει με τον ρόλο του χρήστη και τον ανακατευθύνει στη σελίδα προβολής εκδηλώσεων.

Σελίδα Προβολής διαθέσιμων εκδρομών (Events Page)

Η σελίδα αυτή βρίσκεται στο route `/events`. Χρησιμοποιεί τη μέθοδο `getServerSideProps` του Next.js για να φορτώνει δυναμικά τα δεδομένα από το backend πριν εμφανιστεί η σελίδα στον χρήστη (Server-Side Rendering). Συγκεκριμένα, γίνεται fetch στο endpoint

`http://backend:5000/events` και τα δεδομένα που επιστρέφονται περνάνε ως props στο component της σελίδας.

Στη σελίδα `/events` εμφανίζονται οι διαθέσιμες εκδρομές στην ενότητα `Recommended for you`. Κάθε εκδρομή εμφανίζεται ως κάρτα με εικόνα, τίτλο και τιμή. Η εικόνα κάθε εκδρομής δεν είναι `hardcoded` στον κώδικα, αλλά αντλείται από το πεδίο `preview_image` της βάσης δεδομένων. Για παράδειγμα, αν η βάση έχει την τιμή `/images/knossos.png`, τότε το frontend βρίσκει την εικόνα στον φάκελο `frontend/public/images/knossos.png`. Έτσι, το UI μπορεί να αλλάζει εικόνες χωρίς να χρειάζεται αλλαγή στον κώδικα, αρκεί να τροποποιηθεί το path στη βάση δεδομένων. Η σελίδα διαθέσιμων εκδρομών περιλαμβάνει επίσης την ενότητα `Today's top pick`. Για την ενότητα αυτή χρησιμοποιείται μία από τις εκδρομές που επιστρέφονται από τη βάση, όπου και εμφανίζεται με μεγαλύτερη εικόνα, τίτλο και περιγραφή, ώστε να προβάλλεται ως προτεινόμενη εκδρομή.

Σελίδα Προβολής Εκδρομής (View Event Page)

Η επόμενη σελίδα που υλοποιήθηκε είναι η σελίδα λεπτομερειών της εκδρομής (`Trip/Event Detail`). Η σελίδα αυτή βρίσκεται στο route `/events/[id]`. Το `[id]` αποτελεί δυναμικό route (dynamic routing) του Next.js, οπότε για παράδειγμα η διαδρομή `/events/1` εμφανίζει την εκδρομή με `event_id = 1`.

Η σελίδα χρησιμοποιεί επίσης τη μέθοδο `getServerSideProps` και εκτελεί `fetch` στο endpoint `http://backend:5000/events/{id}`. Με αυτόν τον τρόπο, κάθε φορά που ανοίγει μια σελίδα λεπτομερειών, το frontend ζητάει από το backend τα στοιχεία της συγκεκριμένης εκδρομής. Στη σελίδα λεπτομερειών εμφανίζονται περισσότερες πληροφορίες για την εκδρομή, όπως ο τίτλος, η εικόνα, η περιγραφή, το πρόγραμμα (`schedule`), ο τύπος εκδρομής, η κατάσταση, η ημερομηνία, η ώρα, η τοποθεσία, ο μέγιστος αριθμός συμμετεχόντων, η βαθμολογία και η τιμή. Επίσης, έχουν ενσωματωθεί κουμπιά όπως `Join Event` και `Add to Favourites`, ώστε να αποτυπώνεται οπτικά πώς ο χρήστης μπορεί να δηλώσει συμμετοχή ή να αποθηκεύσει την εκδρομή στα αγαπημένα του.

Σελίδα Προσθήκης Νέας Εκδρομής (Add Event Page)

Η σελίδα αυτή βρίσκεται στο αρχείο `pages/events/add/index.tsx` και χρησιμοποιείται για την καταχώρηση νέων δραστηριοτήτων στην πλατφόρμα από τους `Trip Creators`. Για την υλοποίησή της χρησιμοποιήθηκε η τεχνική του Client-Side Rendering (CSR), καθώς βασίζεται σε μια δυναμική φόρμα συλλογής στοιχείων με άμεση αλληλεπίδραση από τον χρήστη.

Όπως περιγράφηκε στην ενότητα Γ1, η σελίδα αξιοποιεί ένα Grid layout 1x2 (μέσω των κλάσεων `griditem`) για να χωρίσει τη φόρμα σε δύο στήλες. Η αριστερή στήλη περιλαμβάνει τα

πεδία κειμένου Event Title, Location, Price και το component DescriptionField. Η δεξιά στήλη περιλαμβάνει το drop-down μενού επιλογής τύπου εκδρομής (SelectField), το οποίο αντλεί τις τιμές του δυναμικά από το TypeScript enum EventTypes (Other, NightLife, City, Nature, Cultural), τα πεδία για τους συμμετέχοντες, το πρόγραμμα, την ημερομηνία, καθώς και το component ανεβάσματος αρχείου FileUploadField.

Η επικοινωνία με το backend υλοποιείται μέσω της συνάρτησης addEventHandler(), η οποία ενεργοποιείται με το submit της φόρμας. Λόγω της ανάγκης για ταυτόχρονη αποστολή κειμένου και της εικόνας (imageFile), η συνάρτηση βάζει τα δεδομένα σε ένα αντικείμενο FormData μέσω του new FormData(event.currentTarget). Στη συνέχεια ενημερώνεται η κατάσταση (setEventInfo και setSubmit(1)), ενεργοποιώντας το React useEffect. Αυτό καλεί τη συνάρτηση addEvent(), η οποία εκτελεί μια κλήση fetch() με τη μέθοδο POST προς το endpoint <http://localhost:5000/events/add> του backend container μας, χρησιμοποιώντας την ιδιότητα credentials: "include", ώστε να μεταφέρεται το session cookie του Trip Creator.

Το backend δέχεται το FormData, περνάει το αρχείο από ελέγχους (μέγεθος/τύπος μέσω regex) και αποθηκεύει τη νέα εκδρομή στη βάση δεδομένων, επιστρέφοντας το νέο αντικείμενο τύπου EventItem στο frontend. Επειδή είχαμε προβλήματα κατά την υλοποίηση της μεταφοράς του αρχείου που ανεβάζει ο χρήστης στον φάκελο public/images, αποφασίσαμε να κρατήσουμε ένα console.log στο backend για να αποδείξουμε ότι το αρχείο όντως καταφέρνει να περάσει επιτυχώς όλο το data flow του συστήματος (frontend -> backend). Επιπλέον δημιουργήσαμε μια δοκιμαστική εκδρομή (test event) με την εικόνα hike-preview.jpg (η οποία βρίσκεται στο frontend/public/images/hike-preview.jpg), για να δείξουμε επιτυχώς ότι το σύστημα χρησιμοποιεί και αποθηκεύει σωστά τα URLs που αποθηκεύονται στη βάση δεδομένων.